



Lovely Professional University

Department of Computer Science and Engineering

PROJECT REPORT

Cloud-Native DevOps CI/CD Monitoring Platform

Submitted in partial fulfillment of the requirements for the award of the degree of
Bachelor of Technology in Computer Science and Engineering

Submitted By – Arpit Sharma

Reg. No – 12306106

Submitted To – Ms. Arshiya

GitHub repository: [ArpitSharma5691/devops-platform](https://github.com/ArpitSharma5691/devops-platform)

Table of Contents

Table of Contents.....	2
1. Introduction	3
1.1 Application Dashboard.....	3
2. Problem Statement	5
3. Objectives of the Project	6
4. System Architecture	7
5. Tools and Technologies Used	8
5.1 GitHub — Source Code Management	8
6. Docker — Application Containerization	10
6.1 Dockerfile	10
6.2 Docker Desktop — Container Management.....	10
7. Kubernetes — Container Orchestration.....	12
7.1 Kubernetes Cluster — Startup and Dashboard	12
8. Jenkins — CI/CD Automation.....	14
9. Terraform — Infrastructure as Code.....	15
9.1 Terraform Configuration (main.tf).....	15
10. Prometheus — Metrics Collection and Monitoring	17
11. Grafana — Dashboard Visualization	18
12. Complete Project Workflow	19
13. Advantages of the System	20
14. Future Enhancements	21
15. Conclusion	22

1. Introduction

DevOps is a modern software development methodology that integrates development and operations teams to automate, accelerate, and improve the software delivery lifecycle. Traditional software deployment methods involve manual configuration, infrastructure setup, monitoring, and deployment processes — all of which increase deployment time, introduce human errors, and reduce the overall reliability of the system.

This project, the Cloud-Native DevOps CI/CD Monitoring Platform, presents a full end-to-end DevOps automation solution built using industry-standard tools. The project integrates Continuous Integration and Continuous Deployment (CI/CD), containerization, container orchestration, real-time monitoring, and Infrastructure as Code (IaC) into a unified, automated pipeline.

A Node.js web application serves as the deployment target and is containerized using Docker, deployed on a Kubernetes cluster, automated through Jenkins CI/CD pipelines, monitored using Prometheus and Grafana, and provisioned using Terraform.

1.1 Application Dashboard

The following screenshot shows the live DevOps CI/CD Monitoring Platform dashboard running at localhost:3000, displaying real-time Kubernetes cluster metrics including running pods, CPU usage, memory usage, and CI/CD pipeline stage statuses.

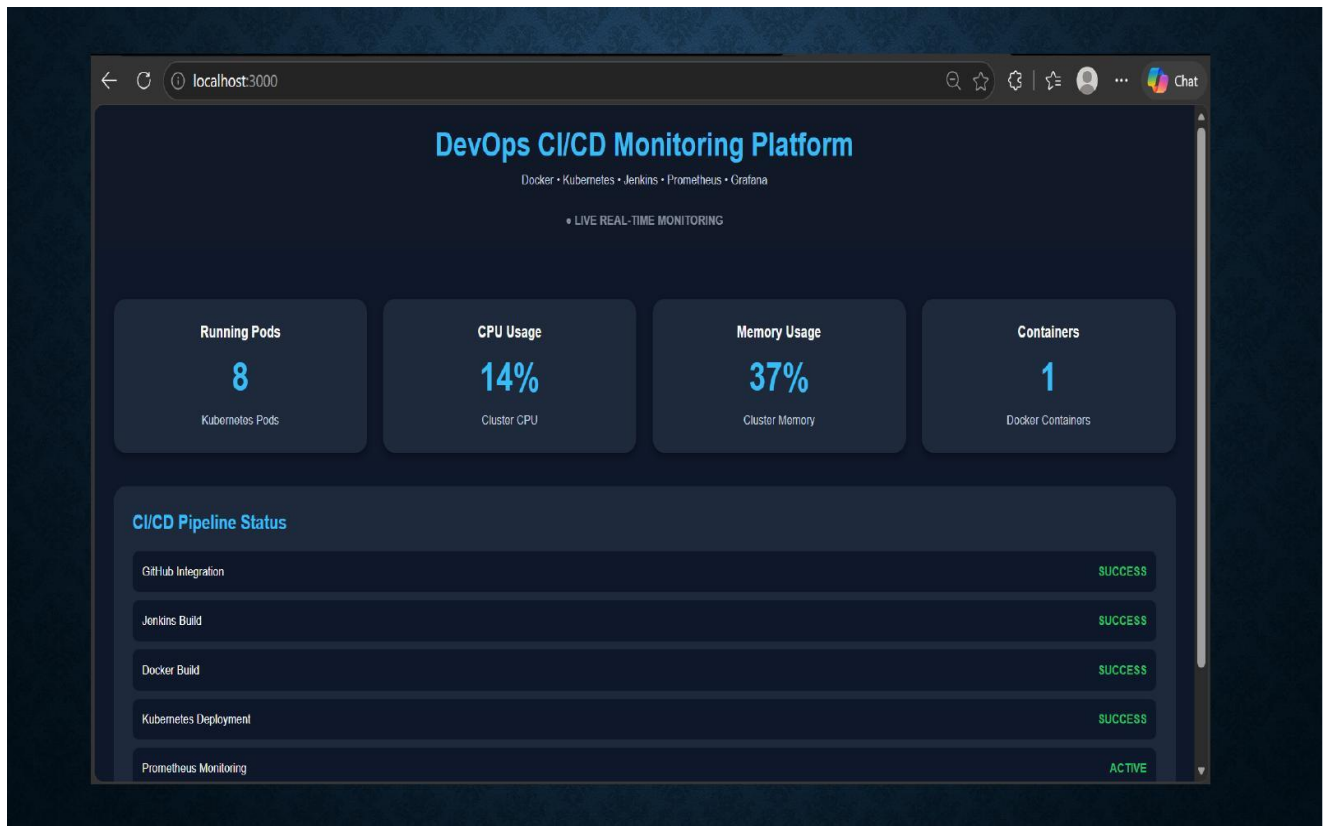


Figure 1.1: DevOps CI/CD Monitoring Platform — Live Dashboard

2. Problem Statement

In traditional software deployment environments, organizations face the following challenges:

- Application deployment is performed manually, resulting in slow release cycles.
- Monitoring systems are limited, making it difficult to detect performance bottlenecks.
- Infrastructure setup is time-consuming and error-prone.
- Scaling applications requires significant manual intervention.
- Deployment failures may cause system downtime affecting end users.
- Manual processes increase the risk of human errors and inconsistencies.
- There is no standardized, repeatable deployment mechanism.

These challenges demand an automated platform that provides:

- Continuous Integration and Continuous Deployment (CI/CD) pipelines.
- Containerized, portable applications.
- Automated real-time monitoring and alerting.
- Infrastructure automation and version-controlled provisioning.
- High availability, scalability, and self-healing deployment capabilities.

This project addresses all of these challenges by implementing a comprehensive DevOps automation platform using Docker, Kubernetes, Jenkins, Prometheus, Grafana, and Terraform.

3. Objectives of the Project

The primary objectives of the Cloud-Native DevOps CI/CD Monitoring Platform are:

- To automate application build, test, and deployment using Jenkins CI/CD pipelines.
- To containerize the Node.js application using Docker for portability and consistency.
- To orchestrate and manage containers at scale using Kubernetes.
- To monitor Kubernetes clusters and application health in real time using Prometheus.
- To visualize system metrics, CPU usage, memory usage, and pod status using Grafana dashboards.
- To automate cloud and local infrastructure provisioning using Terraform (Infrastructure as Code).
- To store and manage application source code and Docker images using GitHub and Docker Hub.
- To implement a scalable, self-healing, and highly available deployment architecture.
- To demonstrate end-to-end real-world DevOps practices applicable to production environments.

4. System Architecture

The system architecture of the Cloud-Native DevOps CI/CD Monitoring Platform is an end-to-end pipeline where every stage of the software delivery lifecycle is automated. The architecture consists of the following layers:

- **Source Control Layer:** GitHub stores and manages the application source code. All code changes trigger the CI/CD pipeline through Jenkins or GitHub Actions integration.
- **CI/CD Layer:** Jenkins detects code changes, builds Docker images, pushes them to Docker Hub, and deploys updated containers to the Kubernetes cluster.
- **Containerization Layer:** Docker packages the Node.js application and all its dependencies into a portable container image.
- **Orchestration Layer:** Kubernetes manages container deployment, scaling, and self-healing. Minikube is used as the local Kubernetes cluster.
- **Monitoring Layer:** Prometheus continuously scrapes metrics from Kubernetes nodes and pods. Grafana displays these metrics through intuitive real-time dashboards.
- **Infrastructure Layer:** Terraform provisions Docker images and containers using Infrastructure as Code, ensuring repeatable and consistent infrastructure setup.

The architecture ensures that every code push by a developer automatically triggers the full pipeline — from build and containerization through deployment, monitoring, and infrastructure provisioning.

5. Tools and Technologies Used

The following table summarizes all tools and technologies integrated into the Cloud-Native DevOps CI/CD Monitoring Platform:

Tool / Technology	Purpose	Version
Docker	Application Containerization	20.x
Kubernetes (Minikube)	Container Orchestration	v1.34.0
Jenkins	CI/CD Automation	2.x
Prometheus	Metrics Collection & Monitoring	Latest
Grafana	Dashboard Visualization	Latest
Terraform	Infrastructure as Code (IaC)	Latest
GitHub	Source Code Management	-
Node.js	Backend Application Runtime	18.x
Express.js	Web Framework	Latest
Docker Hub	Container Image Repository	-

5.1 GitHub — Source Code Management

GitHub is used as the centralized source code hosting and version control platform. The project repository (devops-platform) contains all application source files including app.js, Dockerfile, Kubernetes YAML manifests, the Jenkinsfile pipeline definition, and Terraform configuration files. Jenkins is connected to the GitHub repository to automatically trigger CI/CD pipeline builds on code changes.

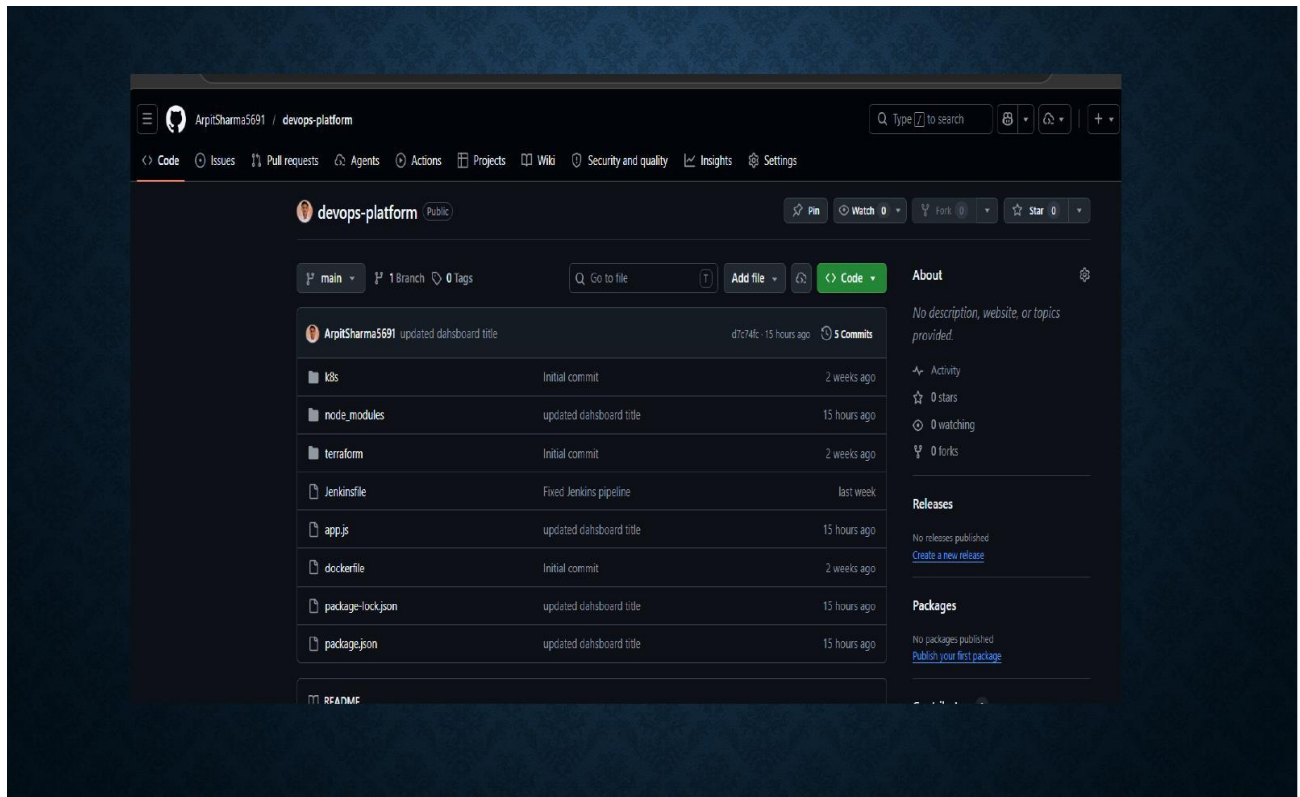


Figure 5.1: GitHub Repository — devops-platform project files

6. Docker — Application Containerization

Docker is a containerization platform that packages an application and all its dependencies — including code, libraries, runtime environment, and configuration — into a single portable container image. Containers ensure that the application runs identically across all environments, whether on a developer's local machine, in a CI/CD pipeline, or in a Kubernetes cluster.

6.1 DockerFile

The following Dockerfile was created for the Node.js application. It uses the official Node.js 18 base image, sets the working directory, copies dependencies, installs them, and exposes port 3000 for the web server.

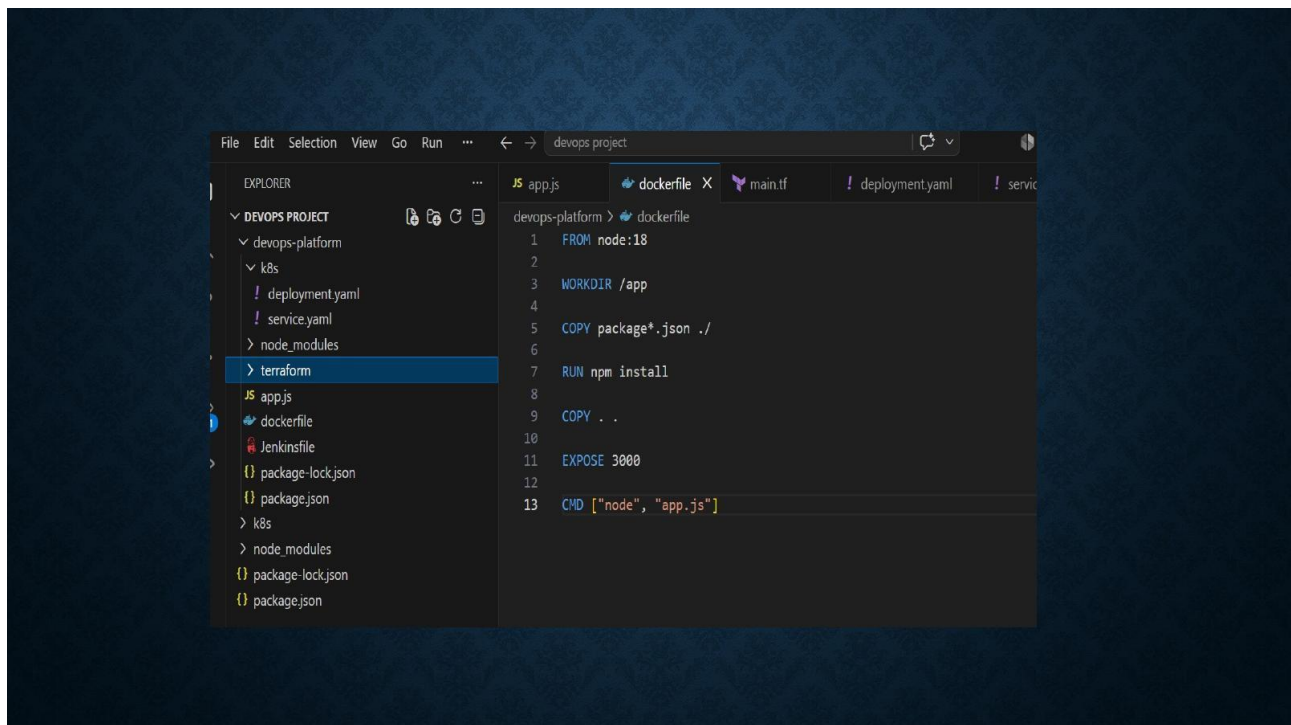


Figure 6.1: Dockerfile — Node.js application containerization configuration

6.2 Docker Desktop — Container Management

Docker Desktop provides a graphical interface to manage running containers, images, volumes, and Kubernetes integration. The screenshot below shows the active Minikube container running on Docker Desktop, consuming 207% CPU across 8 cores and 252.7 MB of memory, confirming that the Kubernetes cluster is operational.

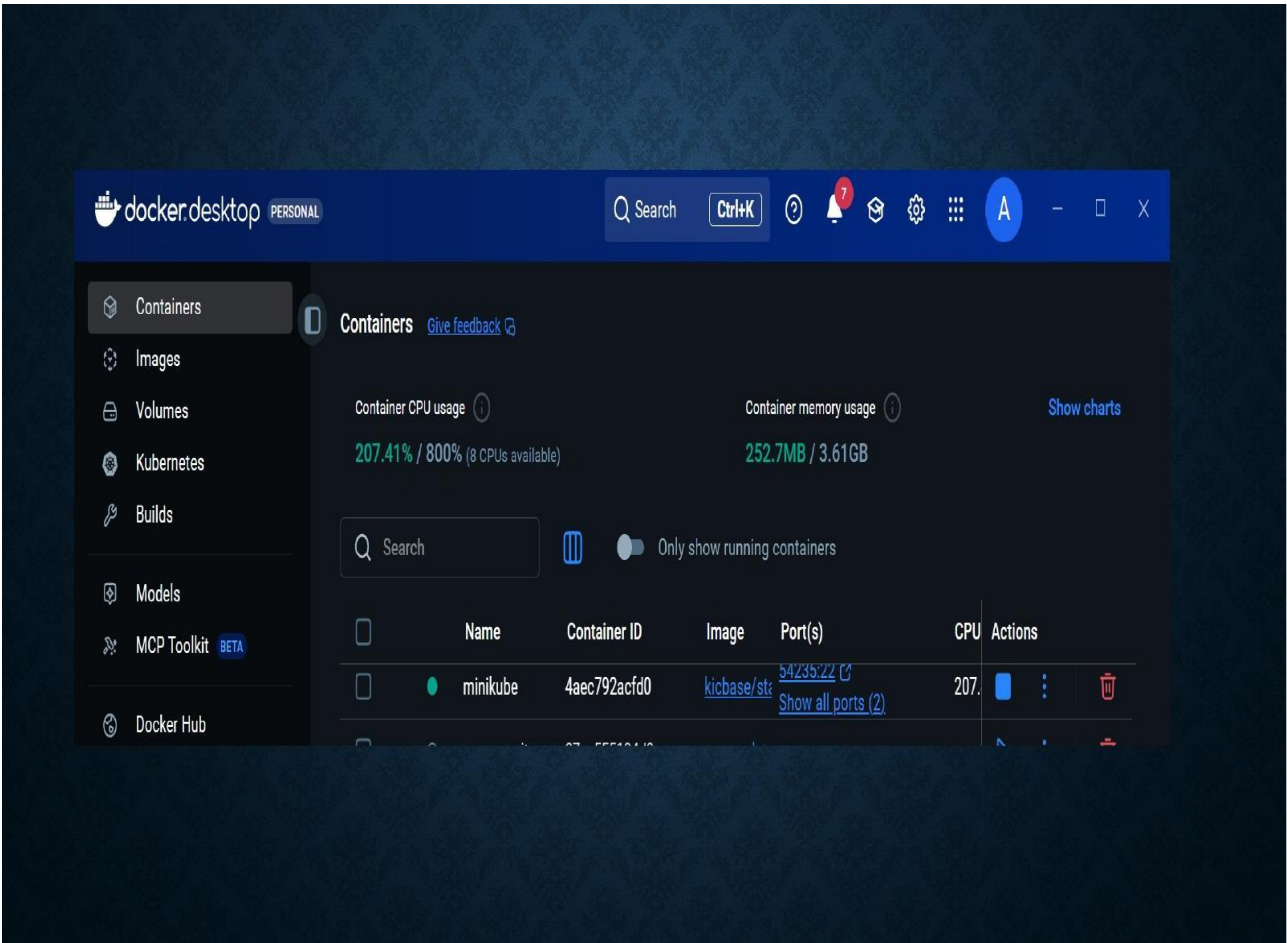


Figure 6.2: Docker Desktop — Active containers including Minikube cluster

7. Kubernetes — Container Orchestration

Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, management, and monitoring of containerized applications. While Docker runs individual containers, Kubernetes manages multiple containers across a cluster, ensuring high availability, load balancing, and self-healing.

In this project, Minikube (v1.37.0) is used as the local Kubernetes cluster running on Windows 11 Home. The cluster is initialized using the Docker driver and hosts all application pods, services, daemon sets, deployments, and replica sets.

7.1 Kubernetes Cluster — Startup and Dashboard

The following screenshots show the Minikube cluster starting up via the PowerShell terminal and the Kubernetes Dashboard displaying the workload status, including 8 running pods, 5 active deployments, and 6 replica sets — all in a healthy running state.

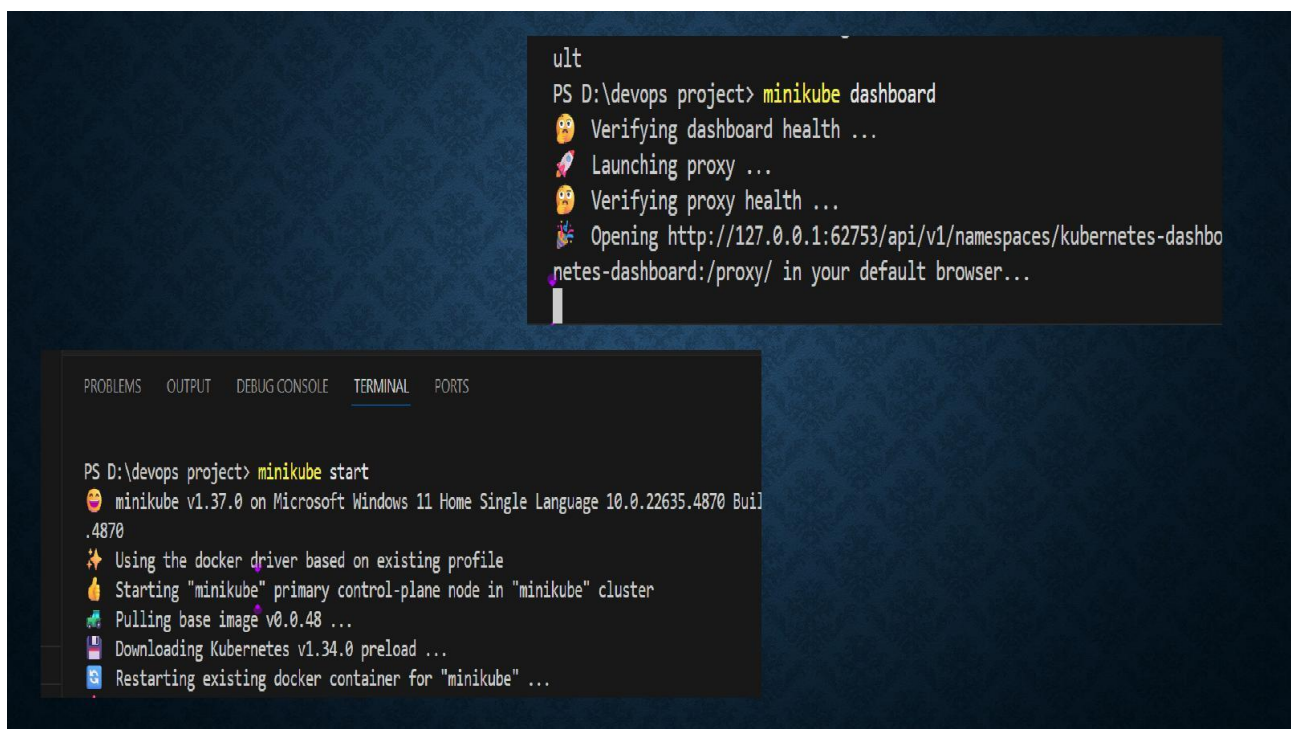


Figure 7.1: Minikube cluster startup — kubectl terminal commands

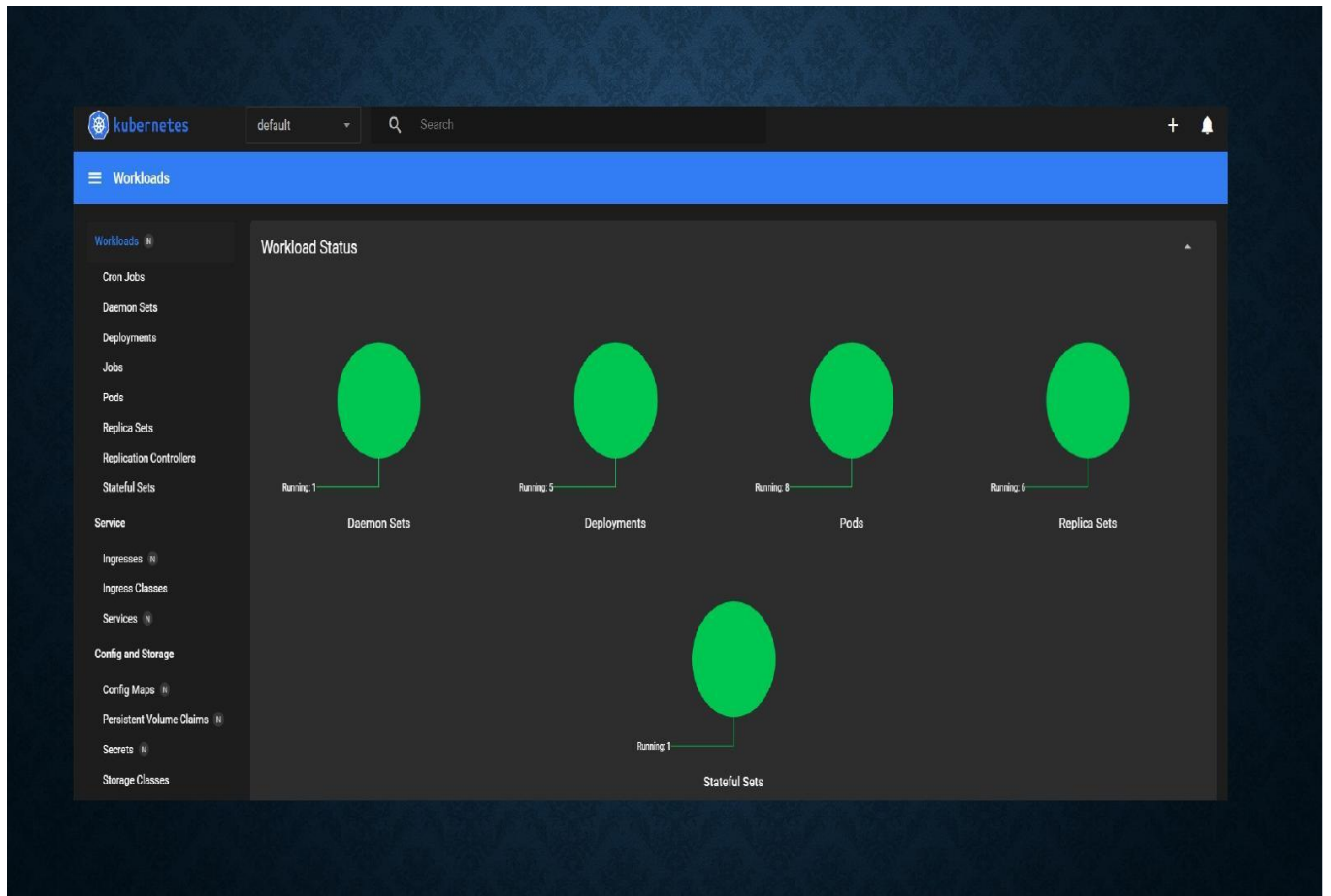


Figure 7.2: Kubernetes Dashboard — Workload Status with all components running

8. Jenkins — CI/CD Automation

Jenkins is an open-source automation server used to implement Continuous Integration (CI) and Continuous Deployment (CD) pipelines. It automates the entire software delivery process from source code checkout to Docker image building to Kubernetes deployment.

The Jenkinsfile defines a declarative pipeline with the following stages:

- Checkout SCM: Jenkins pulls the latest source code from the GitHub repository.
- Build Docker Image: The application is packaged into a Docker image using the Dockerfile.
- Deploy to Kubernetes: The updated Docker image is deployed to the Kubernetes cluster using kubectl.

The screenshot below shows the Jenkins pipeline for build #7, which completed all three stages successfully in 55 seconds. The green checkmarks on each pipeline stage confirm that the Checkout SCM, Build Docker Image, and Deploy to Kubernetes stages all passed without errors.

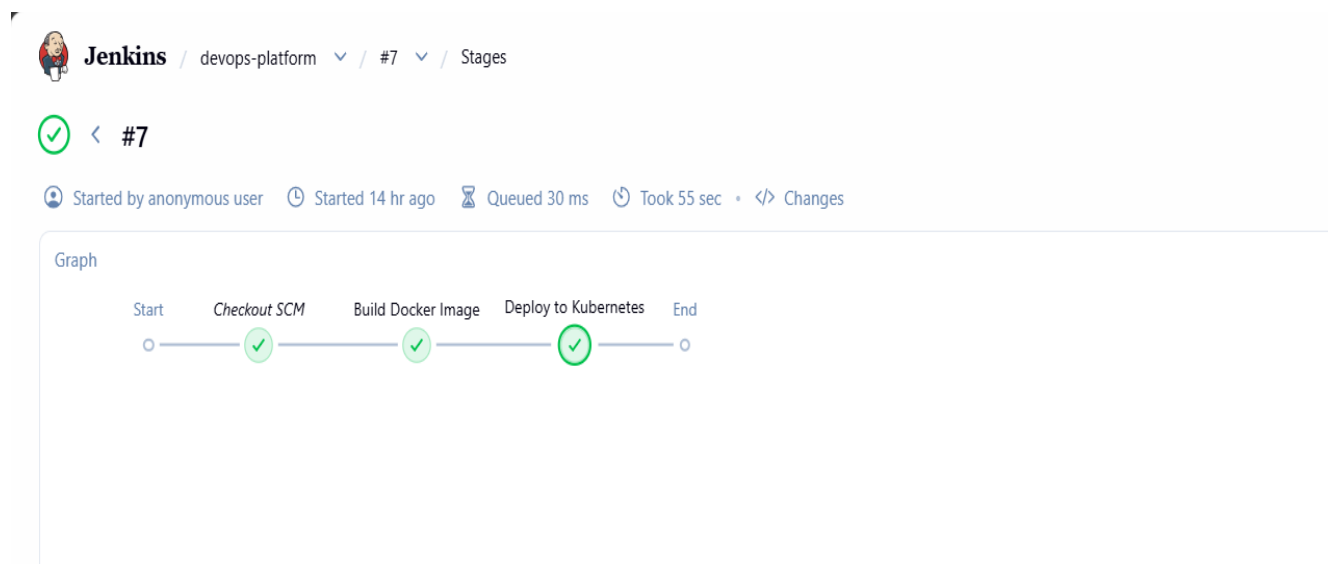


Figure 8.1: Jenkins CI/CD Pipeline — Build #7 with all stages completed successfully

```
devops-platform > Jenkinsfile
1  pipeline {
2
3      agent any
4
5      stages {
6
7          stage('Build Docker Image') {
8              steps {
9                  bat 'docker build -t arpitsharma01/devops-dashboard .'
10             }
11         }
12
13         stage('Deploy to Kubernetes') {
14             steps {
15                 bat 'kubectl apply -f k8s/'
16             }
17         }
18     }
19 }
```

9. Terraform — Infrastructure as Code

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp that allows infrastructure to be defined, provisioned, and managed using human-readable configuration files. It supports a wide variety of cloud providers and infrastructure platforms.

In this project, Terraform uses the Kreuzwerker Docker provider to automate Docker infrastructure — specifically the Docker image pull and container provisioning. This ensures that the infrastructure setup is consistent, repeatable, and version-controlled, following IaC best practices.

9.1 Terraform Configuration (main.tf)

The main.tf configuration file declares the required Docker provider, defines a `docker_image` resource to pull the application image from Docker Hub (arpitsharma01/devops-dashboard), and creates a `docker_container` resource that maps port 3000 internally and externally.

```
devops-platform > terraform > main.tf
1 terraform {
2   required_providers {
3     docker = {
4       source = "kreuzwerker/docker"
5     }
6   }
7 }
8
9 provider "docker" {}
10
11 resource "docker_image" "dashboard" {
12   name = "arpitsharma01/devops-dashboard"
13 }
14
15 resource "docker_container" "dashboard_container" {
16   image = docker_image.dashboard.image_id
17   name  = "devops-dashboard"
18
19   ports {
20     internal = 3000
21     external = 3000
22   }
23 }
```

Figure 9.1: Terraform main.tf — Docker provider configuration for IaC provisioning

10. Prometheus — Metrics Collection and Monitoring

Prometheus is an open-source systems monitoring and alerting toolkit. It collects metrics by scraping HTTP endpoints exposed by applications and infrastructure components at regular intervals. In this project, Prometheus is deployed within the Kubernetes cluster to continuously collect metrics from:

- Kubernetes API Server — cluster-level metrics.
- Node Exporter — hardware and OS-level metrics (CPU, memory, disk, network) from Kubernetes nodes.
- Application pods — custom application metrics.

The screenshot below shows the Prometheus Target Health page, confirming that the kubernetes-api-servers target endpoint (<https://192.168.49.2:8443/metrics>) is successfully scraped with an UP state, with a last scrape duration of 341ms.

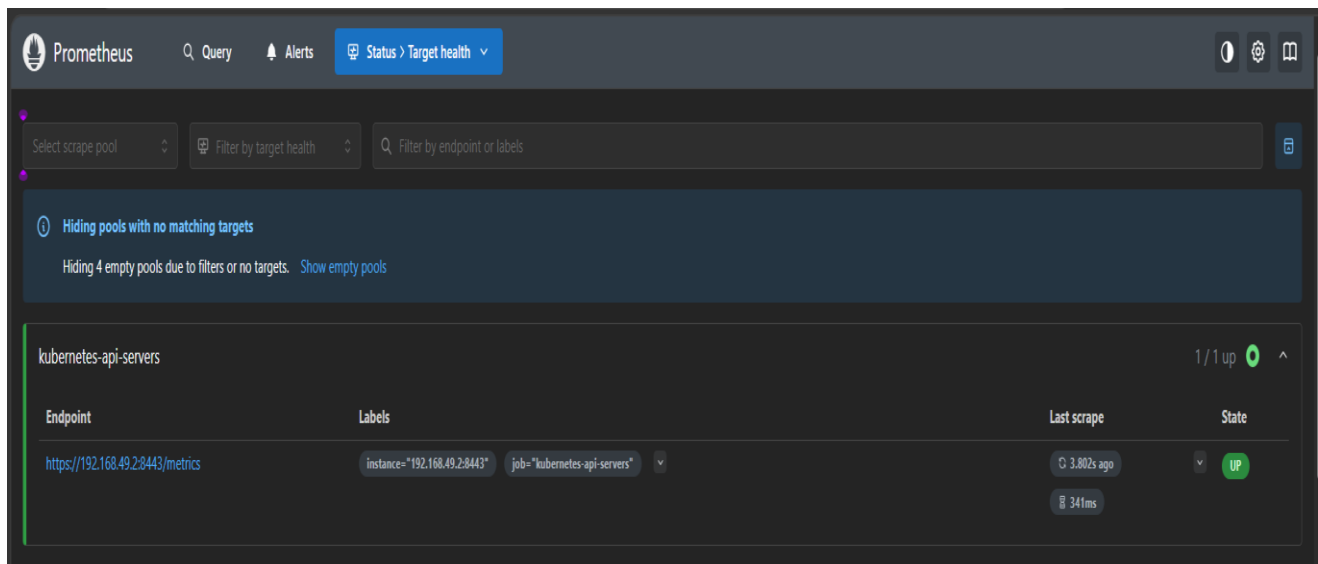


Figure 10.1: Prometheus — Target health showing Kubernetes API server scraping UP

11. Grafana — Dashboard Visualization

Grafana is an open-source analytics and interactive visualization platform. It connects to Prometheus as a data source and renders the collected metrics as rich, real-time dashboards with graphs, gauges, and charts.

In this project, the Node Exporter dashboard in Grafana provides the following real-time system metrics from the Minikube Kubernetes node:

- CPU Busy: 9.8% — percentage of CPU currently utilized.
- System Load: 5.9% — overall system load average.
- RAM Used: 46.9% — memory consumption of the cluster node.
- SWAP Used: 0.2% — minimal swap space usage.
- CPU Core Count: 8 cores available.
- Total RAM: 4 GiB.
- Uptime: 19.0 minutes.

The Grafana dashboard is configured with Prometheus as the data source, the job set to kubernetes-service-endpoints, and the node name set to minikube. The dashboard refreshes every 1 minute and displays data from the last 24 hours.



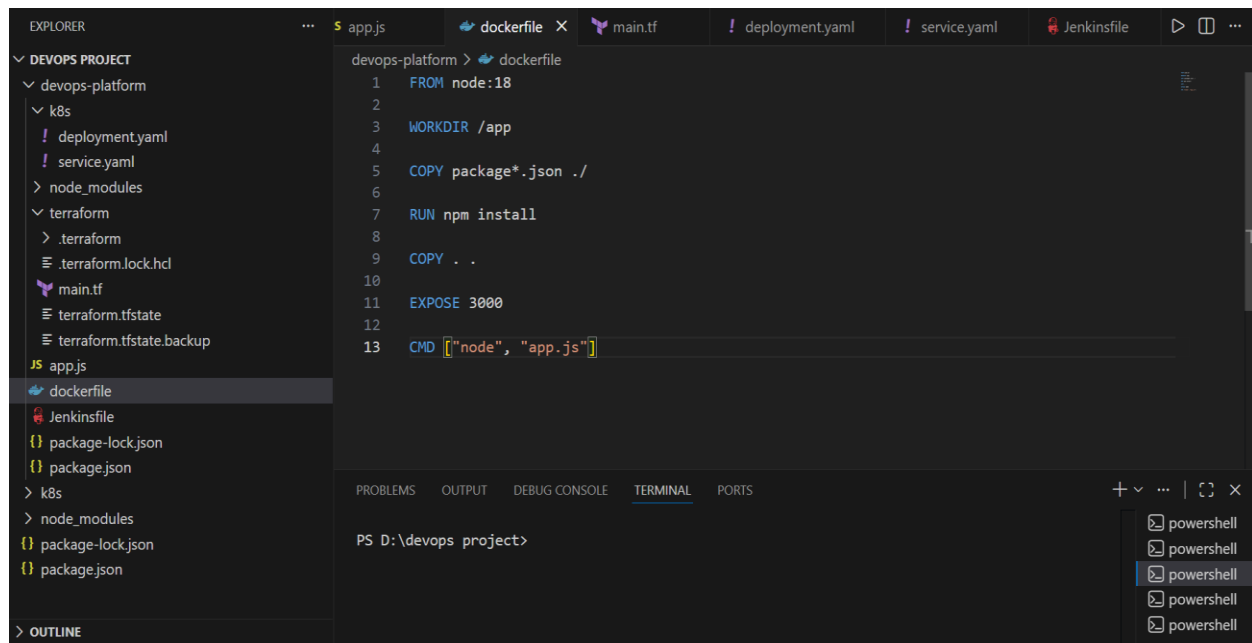
Figure 11.1: Grafana Node Exporter Dashboard — Real-time CPU, memory, and system metrics

12. Complete Project Workflow

The end-to-end workflow of the Cloud-Native DevOps CI/CD Monitoring Platform is structured as follows:

#	Phase	Description
Step 1	Application Development	A Node.js web application built with Express.js serves as the base application for the platform.
Step 2	Containerization	The application is containerized using Docker by writing a Dockerfile and building a Docker image.
Step 3	Docker Hub Integration	The built Docker image is pushed to Docker Hub for centralized storage and distribution.
Step 4	Kubernetes Deployment	Kubernetes deploys the containerized application using Deployment and Service YAML manifests.
Step 5	CI/CD Pipeline	Jenkins automates the entire build, push, and deployment process through a declarative pipeline.
Step 6	Monitoring Setup	Prometheus scrapes metrics from the Kubernetes cluster and Grafana visualizes them in real-time dashboards.
Step 7	Infrastructure Automation	Terraform uses the Kreuzwerker Docker provider to automate Docker image and container provisioning via IaC.

This workflow ensures that every code change goes through a standardized, automated pipeline — from development through containerization, deployment, and monitoring — with no manual intervention required at any stage.



13. Advantages of the System

- Eliminates manual deployment effort, reducing human errors and deployment time.
- Accelerates software delivery speed through fully automated CI/CD pipelines.
- Enables scalable and self-healing container deployments using Kubernetes.
- Provides continuous, real-time monitoring and visibility into cluster health and performance.
- Reduces system downtime through Kubernetes self-healing and automated rollback capabilities.
- Simplifies and standardizes infrastructure management through Terraform IaC.
- Improves deployment reliability and consistency across all environments.
- Demonstrates and validates industry-standard real-world DevOps practices.
- Makes it easy to onboard new team members with fully automated, documented pipelines.
- Provides a foundation for advanced features such as auto-scaling and multi-cluster deployments.

14. Future Enhancements

The following features and improvements are planned for future iterations of the platform:

- Auto-scaling based on real-time traffic load using Kubernetes Horizontal Pod Autoscaler (HPA).
- Alerting and notification system using Prometheus AlertManager with Slack and email integration.
- Load balancing using Nginx Ingress Controller or AWS Elastic Load Balancer.
- Multi-node Kubernetes cluster setup using Amazon EKS (Elastic Kubernetes Service) for production-grade deployments.
- Centralized logging pipeline using the ELK Stack (Elasticsearch, Logstash, Kibana).
- Advanced security integration including RBAC policies, network policies, and secret management.
- GitOps deployment model using ArgoCD or Flux for declarative, Git-driven deployments.
- Multi-environment pipeline support (development, staging, production) with environment-specific configurations.
- Terraform remote state management using AWS S3 and DynamoDB for state locking.
- Integration with GitHub Actions for cloud-native CI/CD without a separate Jenkins server.

15. Conclusion

The Cloud-Native DevOps CI/CD Monitoring Platform successfully demonstrates the design and implementation of a complete, end-to-end DevOps automation pipeline using industry-standard tools and technologies.

The project integrates Docker for application containerization, Kubernetes for container orchestration and self-healing deployment, Jenkins for automated CI/CD pipeline execution, Prometheus for real-time metrics collection, Grafana for dashboard-based monitoring and visualization, and Terraform for Infrastructure as Code-based provisioning.

The platform automates the entire software delivery lifecycle — from source code management in GitHub through Docker image building and Docker Hub storage, to Kubernetes deployment, and continuous real-time monitoring. All pipeline stages have been verified and demonstrated through actual screenshots, confirming successful execution of each component.

This project provides a practical, hands-on demonstration of modern cloud-native DevOps practices that are directly applicable to real-world enterprise software development environments. The architecture is designed to be extensible, enabling future enhancements such as auto-scaling, centralized logging, advanced alerting, and multi-cloud deployments.